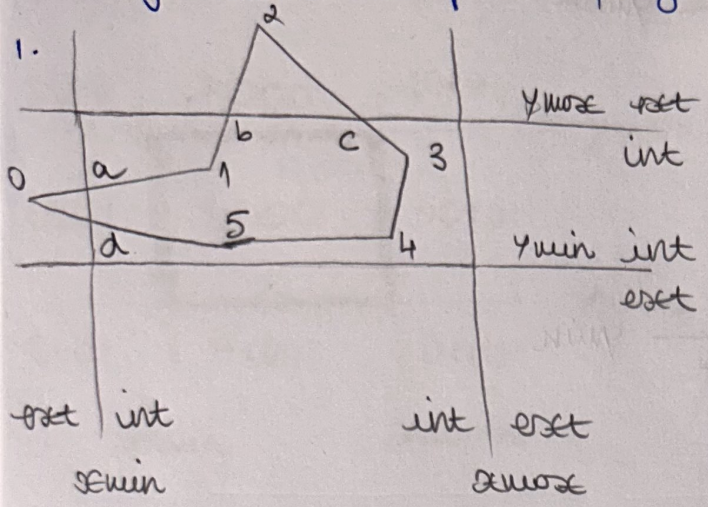
Algoritmul Sutherland-Hodgman

↳ algoritm de decupare a poligoanelor

1.



1. ymax

0-1 : 0,1 → 0,1

1-2 : 1,b → 0,1,1,b

2-3 : c,3 → 0,1,1,b,c,3

3-4 : 3,4 → 0,1,1,b,c,3,3,4

4-5 : 4,5 → 0,1,1,b,c,3,3,4,4,5

5-0 : 5,0 → 0,1,1,b,c,3,3,4,4,5,5,0

lista : 0,1,b,c,3,4,5

2. xmax

0-1 : 0,1 → 0,1

1-b : 1,b → 0,1,1,b

b-c : b,c → 0,1,1,b,b,c

c-3 : c,3 → 0,1,1,b,b,c,c,3

3-4 : 3,4 → 0,1,1,b,b,c,c,3,3,4

4-5 : 4,5 → 0,1,1,b,b,c,c,3,3,4,4,5

5-0 : 5,0 → 0,1,1,b,b,c,c,3,3,4,4,5,5,0

lista : 0,1,b,c,3,4,5

3. ymin

0-1 : 0,1 → 0,1

1-b : 1,b → 0,1,1,b

b-c : b,c → 0,1,1,b,b,c

c-3 : c,3 → 0,1,1,b,b,c,c,3

3-4 : 3,4 → 0,1,1,b,b,c,c,3,3,4

4-5 : 4,5 → 0,1,1,b,b,c,c,3,3,4,4,5

5-0 : 5,0 → 0,1,1,b,b,c,c,3,3,4,4,5,5,0

lista : 0,1,b,c,3,4,5

4. xmin

0-1 : a,1 → a,1

1-b : 1,b → a,1,1,b

b-c : b,c → a,1,1,b,b,c

c-3 : c,3 → a,1,1,b,b,c,c,3

3-4 : 3,4 → a,1,1,b,b,c,c,3,3,4

4-5 : 4,5 → a,1,1,b,b,c,c,3,3,4,4,5

5-0 : 5,d → a,1,1,b,b,c,c,3,3,4,4,5,5,d

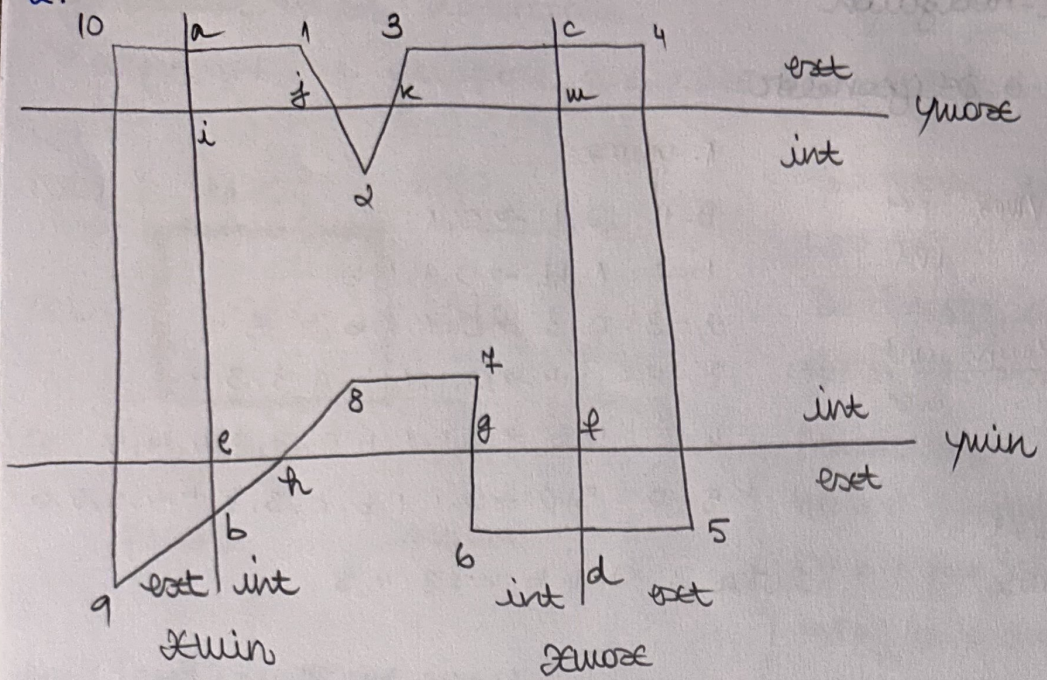
lista : a,1,b,c,3,4,5,d

⇒ lista finală :

a,1,b,c,3,4,5,d

↳ vârfurile ~~poligonului~~ poligonului decupat

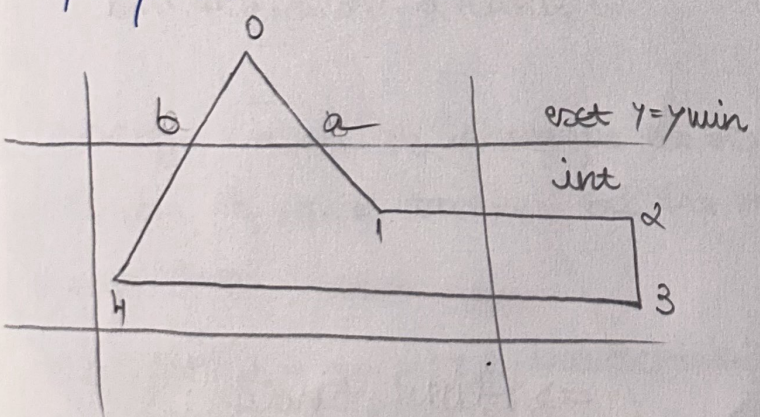
2.



1. ~~xmin~~ : a, 1, 2, 3, 4, 5, 6, 7, 8, b
2. ~~xmax~~ : a, 1, 2, 3, c, d, b, 7, 8, b
3. ymin : e, a, 1, 2, 3, c, g, 7, 8, h
4. ymax : e, i, j, 2, k, m, f, g, 7, 8, h

3. poligon cu laturile [0,1), [1,2), [2,3), [3,4), [4,0)

Vârfurile poligonului rezultat în etapa decupării față de dreapta y = ymin



0-1 : a, 1 → a, 1

1-2 : 1, 2 → a, 1, 1, 2

2-3 : 2, 3 → a, 1, 1, 2, 2, 3

3-4 : 3, 4 → a, 1, 1, 2, 2, 3, 3, 4

4-0 : 4, b → a, 1, 1, 2, 2, 3, 3, 4, 4, b

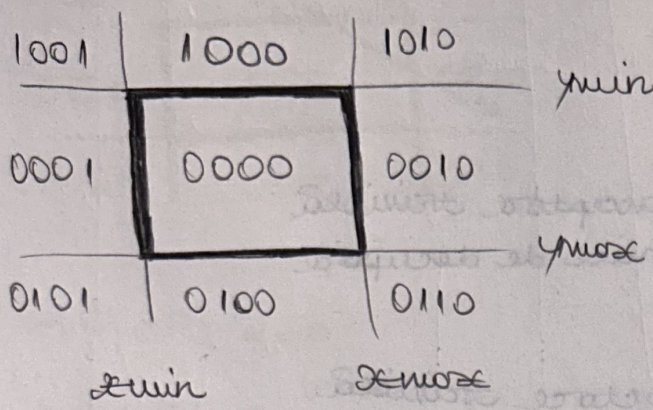
listă finală : a, 1, 2, 3, 4, b

Vârfurile rezultate în urma decupării

Algoritmul Cohen-Sutherland

↳ algoritm de decupare a liniilor

sus jos dreapta stânga
$b_3\ b_2\ b_1\ b_0$



· punctele din plan sunt codificate pe 4 biți $b_3b_2b_1b_0$, în funcție de poziția lor față de dreptunghiul de decupare

convenția

$b_0 = 1$ punctele din stânga dreptunghiului
$b_1 = 1$ punctele din dreapta dreptunghiului
$b_2 = 1$ sub dreptunghi
$b_3 = 1$ deasupra dreptunghiului

⇓

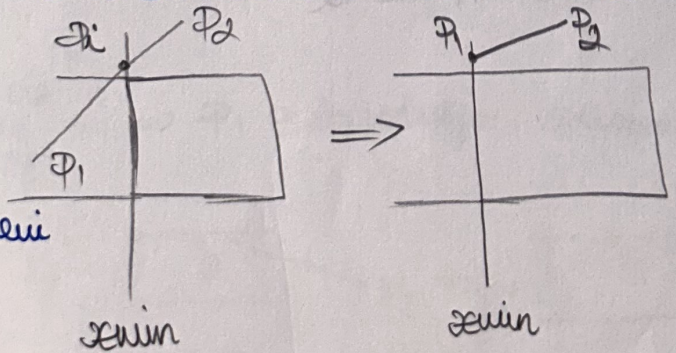
$b_0 = 1$ $x < x_{min}$
$b_1 = 1$ $x > x_{max}$
$b_2 = 1$ $y > y_{max}$
$b_3 = 1$ $y < y_{min}$

dacă (cod1 == 0 && cod2 == 0) {
segment acceptat trivial
}
altfel
dacă ((cod1 & cod2) != 0) {
segment rejectat trivial
(cele 2 coduri au același bit 1)
P_1 și P_2 de aceeași parte a dreptunghiului
⇒ segmentul nu intersectează dreptunghiul
}
altfel: se intersectează segmentul
P_1-P_2 cu dreptunghiul de decupare

· se alege latura de intersecție cu segmentul P_1-P_2 pe baza codului lui P_1, care trebuie să fie situat în afara dreptunghiului de decupare

· dacă cod1 == 0 ⇒ se interschimbă P_1 cu P_2

cod1 & 0001 != 0
⇒ P_1 în stânga dreptunghiului



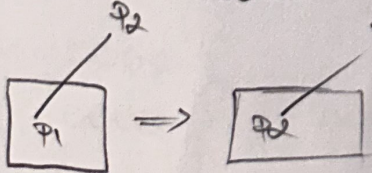
· se calculează intersecția $x = x_{min}$

· se mută P_1 în punctul de intersecție

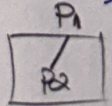
· în iterația următoare

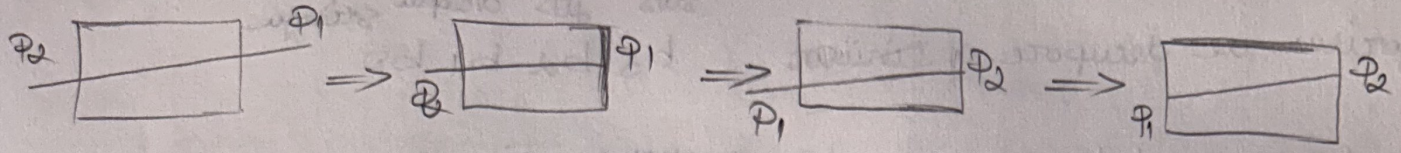
cod1 & cod2 != 0
⇒ segmentul este rejectat

cod1 = 0



cod1 & 8 != 0
P_1 deasupra dreptunghiului





Pași algoritm

1) dacă cod1 == 0 și cod2 == 0 ⇒ acceptare trivială
↳ ambele puncte sunt în dreptunghiul de decupare

2) altfel
↳ dacă cod1 & cod2 != 0 ⇒ rejectare trivială
P1 și P2 sunt de aceeași parte a dreptunghiului

3) altfel (dacă cod1 & cod2 == 0)
verificăm în ce parte a dreptunghiului de decupare e P1

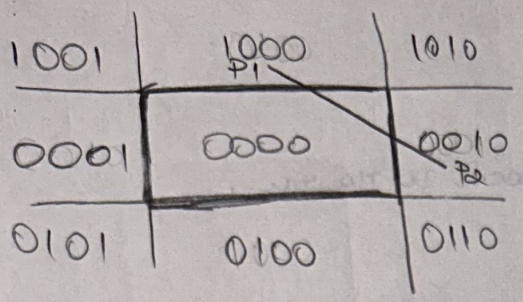
cod1 & 1 != 0 stânga 0001
cod1 & 2 != 0 dreapta 0010
cod1 & 4 != 0 sus 0100
cod1 & 8 != 0 deasupra 1000

→ ca să știm cu ce prelungire a dreptunghiului de decupare se intersectează

↳ calculăm punctul de intersecție și mutăm P1 în Pi, după reluăm verificările, schimbându-l pe P1 cu P2

cod1 == 0
⇓
e în dreptunghiul de decupare ⇒ schimbăm P1 cu P2

Exercitiu



$P_1 = 1000 \neq 0000$

$P_2 = 0010 \neq 0000$

$$P_1 \& P_2 = \begin{array}{r} 1000 \& \\ 0010 \\ \hline 0000 \end{array}$$

$\Rightarrow P_1 \& P_2 == 0$

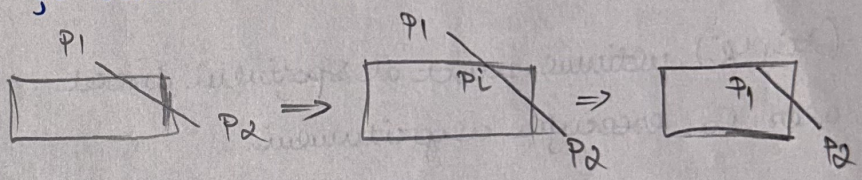
$$cod1 \& 1 = \begin{array}{r} 1000 \& \\ 0001 \\ \hline 0000 \end{array} = 0$$

$$cod1 \& 2 = \begin{array}{r} 1000 \& \\ 0010 \\ \hline 0000 \end{array} = 0$$

$$cod1 \& 4 = \begin{array}{r} 1000 \& \\ 0100 \\ \hline 0000 \end{array} = 0$$

$$cod1 \& 8 = \begin{array}{r} 1000 \& \\ 1000 \\ \hline 1000 \end{array} = 1$$

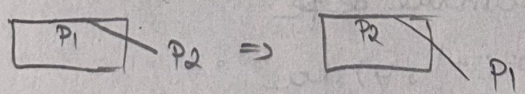
$\Rightarrow P_1$ se afla deasupra dreptunghiului de decupare $\Rightarrow$ calculăm intersecția P_i și scriem P_i cu P_1



avem $P_1 = 0000$, $P_2 = 0010$ $\Rightarrow P_1 \& P_2 = 0$

$P_1 = 0000 \Rightarrow$ e în dreptunghi

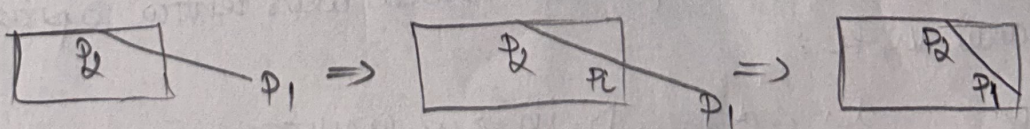
$\rightarrow$ scriem P_1 și P_2



$P_1 = 0010$

$P_2 = 0000$

$$cod1 \& 2 = \begin{array}{r} 0010 \& \\ 0010 \\ \hline 0010 \end{array} = 1 \Rightarrow P_1 \text{ e în ~~stânga~~ dreapta}$$



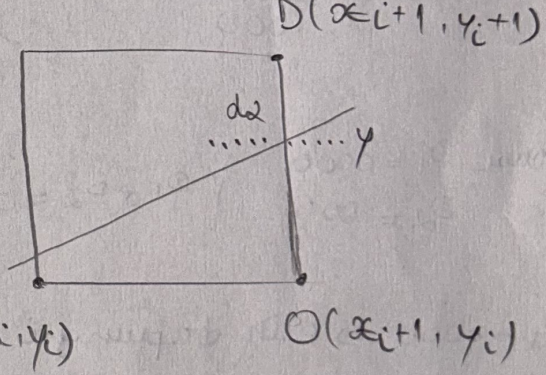
avem $P_1 = 0000$, $P_2 = 0000$, $P_1 \& P_2 = 0$ $\Rightarrow$ acceptare trivială

Algoritmul Bresenham

- Conține numai operații cu numere întregi
- Este definit pentru vectori din primul octan
  - ↳ un vector face parte din primul octan dacă panta sa, m, satisface condiția $0 < m < 1$
- coordonatele pixelilor de pe traseul vectorului se obțin prin calcul incremental
- pentru fiecare valoare a lui x, de la ~~x~~ x_min_vector la ~~x_max~~ x_max_vector, se alege acel punct al spațiului discret care este mai aproape de punctul de pe vectorul teoretic.

- fie $m = \frac{y_2 - y_1}{x_2 - x_1}$ panta vectorului

- (x_i, y_i) ultimul punct al spațiului discret ales la execuția algoritmului



- d_1 = distanța de la punctul de pe vectorul teoretic, (x_{i+1}, y) la punctul $O(x_{i+1}, y_i)$
- d_2 = distanța de la punctul de pe vectorul teoretic (x_{i+1}, y) la punctul $D(x_{i+1}, y_i + 1)$
- O, D adrese de pixeli (puncte ale spațiului discret)
- următorul punct al spațiului discret ales pentru aproximarea vectorului va fi:
  - O - $d_1 < d_2$, D în rest contrar
  - dacă $d_1 = d_2$ se poate alege oricare dintre cele 2 puncte